Bacharelado em Ciência da Computação
Métodos Heurísticos

Prof. Diego Mello da Silva

Instituto Federal de Minas Gerais - Campus Formiga

5 de dezembro de 2014

Sumário

1 GRASP

2 Path Relinking

3 Hibridização GRASP + Path Relinking

4 Referências Bibliográficas

GRASP

Fundamentos

- Heurística GRASP é um processo iterativo ou multi-partida para problemas combinatórios (Feo e Resende, 1989)
- Cada iteração do GRASP compreende duas fases:
  (1) Fase Construtiva: constrói uma solução factível
  (2) Fase de Busca Local: investiga vizinhança até encontrar ótimo local
- GRASP: *Greedy Randomized Adaptative Search Procedure*
  (a) Greedy

  Fase construtiva considera a construção de uma solução factível onde elementos são incrementados à solução em construção de acordo com uma função de avaliação gulosa

  (b) Randomized

  Dentre os melhores candidatos (que resultam em menor custo incremental), escolhe aleatoriamente um deles para incorporar à solução parcial em construção

  (c) Adaptative

  Atualiza lista de candidatos e recalcula os custos incrementais

Pseudo-Código: GRASP

Algoritmo 1 GRASP(*MaxIter*, *Seed*)

```
1: ReadInput()
2: for (k ← 1 to MaxIter) do
3:     Solution ← GreedyRandomizedConstruction(Seed)
4:     Solution ← LocalSearch(Solution)
5:     UpdateSolution(Solution, BestSolution)
6: end for
7: return BestSolution
```

GRASP: Fase Construtiva

- Conjunto de elementos candidatos é formado por todos os elementos que podem ser incorporados à solução parcial sobre construção sem destruir a factibilidade
- A seleção do próximo elemento candidato para incorporação é determinado pela avaliação de todos os elementos candidatos segundo uma função de avaliação gulosa
- Ela geralmente representa o aumento incremental no custo ao incorporar o elemento na solução em construção
- A avaliação dos elementos por essa função leva à construção da Lista de Candidatos Restritos (RCL) formada pelos melhores elementos, i.e., aqueles cuja incorporação à solução leva aos menores custos incrementais (**aspecto guloso**)
- O elemento a ser incorporado na solução parcial é selecionado aleatoriamente da RCL (**aspecto probabilístico**)
- Uma vez que o elemento é incorporado na solução parcial, a lista de candidatos é atualizada e os custos incrementais são reavaliados (**aspecto adaptativo**)

Pseudo-Código: Fase Construtiva

Algoritmo 2 GreedyRandomizedConstruction(*Seed*)

1: **`Solution`** $\leftarrow \varnothing$
2: Avaliar o custo incremental dos elementos candidatos
3: **while** (**`Solution`** não for uma solução completa) **do**
4: Construir a lista de candidatos restrita (RCL)
5: Selecionar um elemento s da RCL aleatoriamente
6: **`Solution`** $\leftarrow$ **`Solution`** $\cup\ s$
7: Reavaliar os custos incrementais
8: **end while**
9: **return `Solution`**

GRASP: Fase Busca Local

- As soluções geradas pela construção gulosa aleatorizada não são necessariamente ótimas
- Fase de busca local geralmente melhora a solução construída
- Esta fase funciona de maneira iterativa, substituindo sucessivamente a solução corrente por uma solução melhor que a corrente contida na sua vizinhança
- Efetividade do procedimento de busca local depende de vários aspectos:
  - Estrutura da vizinhança
  - Técnica de busca na vizinhança: *best-improving* ou *first-improving*
  - Avaliação rápida da função de custo dos vizinhos
  - Solução inicial da busca (fase construtiva deve construir soluções de alta qualidade para a busca local)
- Segundo Resende e Ribeiro (em [Glover]), tem sido observado que convergência prematura é mais provável de ocorrer com *best-improvement*

Pseudo-Código: Fase Busca Local

Algoritmo 3 LocalSearch(Solution)

1: **while** (**`Solution`** não for localmente ótima) **do**
2: Encontrar $s' \in N(\texttt{Solution})$ com $f(s') < f(\texttt{Solution})$
3: **`Solution`** $\leftarrow s'$
4: **end while**
5: **return `Solution`**

Construção da RCL

- GRASP é simples e fácil de implementar; poucos parâmetros precisam ser ajustados
  - Critério de Parada
  - Qualidade dos elementos na RCL
- GRASP pode ser visto como uma técnica de amostragem repetitiva
- Cada iteração produz uma solução amostrada a partir de uma distribuição desconhecida, cuja média e variância são funções da natureza restritiva da RCL
  - RCL com único elemento: variância nula e média igual ao valor da solução gulosa
  - RCL com mais elementos: maior variância, média 'pior', embora melhor solução encontrada supéra a média e frequentemente é ótimo
- Observar histogramas do custo incremental em função do α, em Resende e Ribeiro ([Glover])

Construção da RCL

Algoritmo 4 GreedyRandomizedConstruction(α, *Seed*)

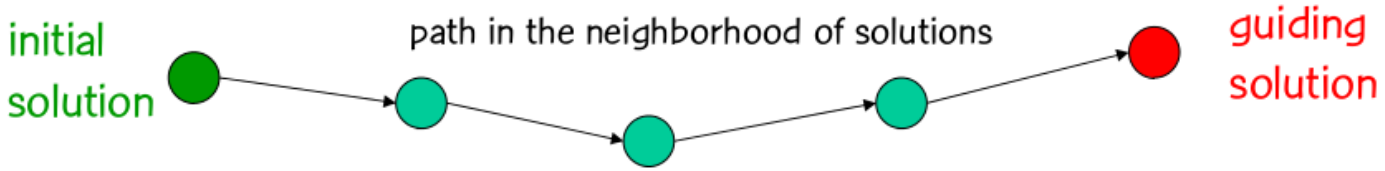
1: **`Solution`** $\leftarrow \varnothing$
2: Inicializa o conjunto candidato: $C \leftarrow E$
3: Avalia custo incremental $c(e)$ para cada $e \in C$
4: **while** ($C \neq \varnothing$) **do**
5: $\quad c^{\min} \leftarrow \min\{c(e) \mid e \in C\}$
6: $\quad c^{\max} \leftarrow \max\{c(e) \mid e \in C\}$
7: $\quad RCL \leftarrow \{e \in C \mid c(e) \leq c^{\min} + \alpha(c^{\max} - c^{\min})\}$
8: $\quad$ Selecione um elemento $s \in RCL$ aleatoriamente
9: $\quad$ **`Solution`** $\leftarrow$ **`Solution`** $\cup\{s\}$
10: $\quad$ Atualiza o conjunto candidato C
11: $\quad$ Reavalia o custo incremental $c(e)$ para todo $e \in C$
12: **end while**
13: **return `Solution`**

- $c^{\min}$ e $c^{\max}$: menor e maior custo incremental, respectivamente
- Quando $\alpha = 0$, construção é puramente gulosa
- Quando $\alpha = 1$, construção é aleatória

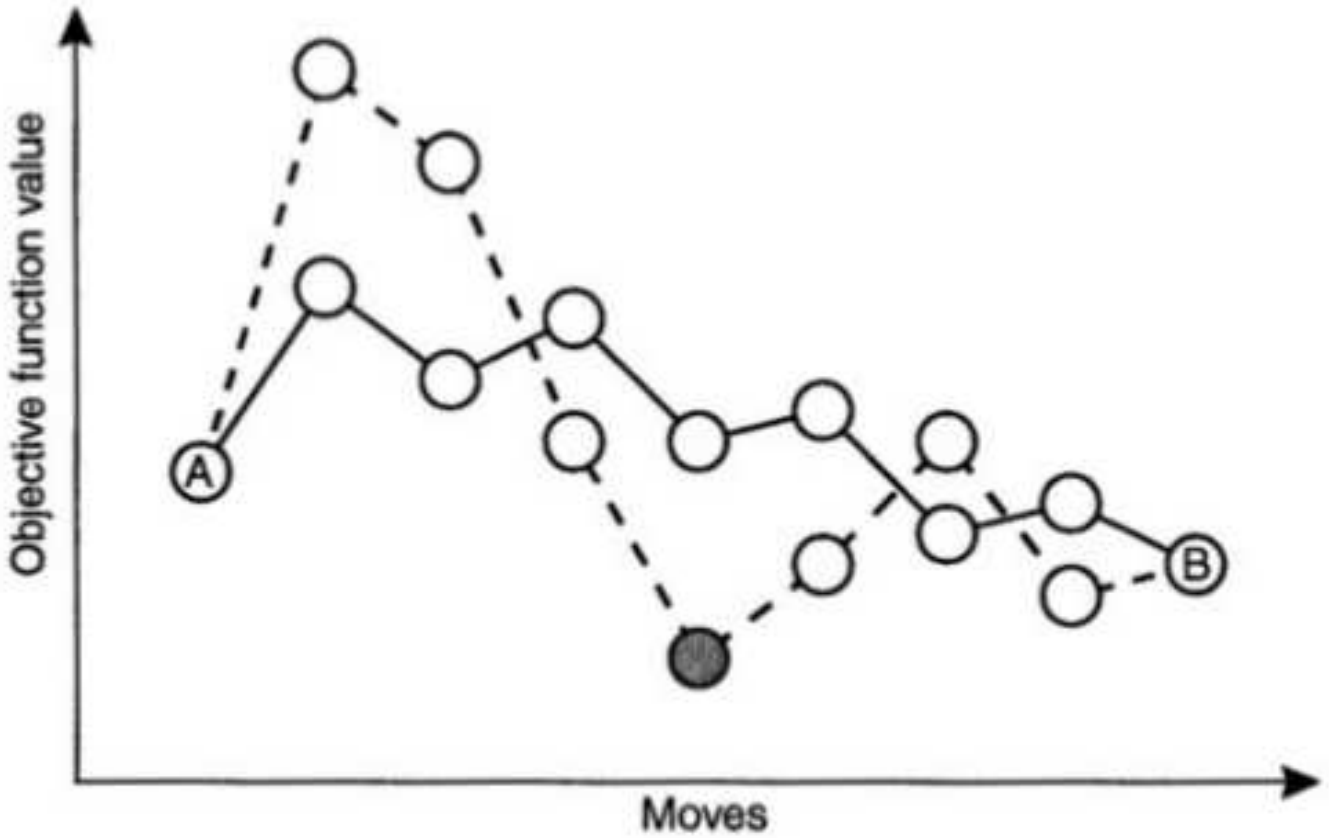
Path Relinking

Path Relinking

- Proposto inicialmente por (Glover e Laguna, 1997) para a fase de diversificação da heurística Scatter Search e Busca Tabu
- Gera novas soluções ao se explorar o trajeto que conecta soluções de boa qualidade
- Princípio de Funcionamento: soluções de boa qualidade (**soluções elite**) sempre têm boas características a serem compartilhadas
- Funcionamento básico:
  - Procedimento exploratório seleciona **solução inicial**
  - Constrói um caminho pelo espaço de soluções (vizinhança) que conecta a solução inicial a outra solução, denominada **solução guia**
  - A cada movimento realizado em direção à solução guia, gera novas soluções intermediárias
  - Movimento realizado insere características ou atributos da solução guia na solução intermediária
- Sequência pode encontrar solução vizinha melhor que as já conhecidas, a um ótimo local nunca antes explorado e, eventualmente, a um ótimo global.

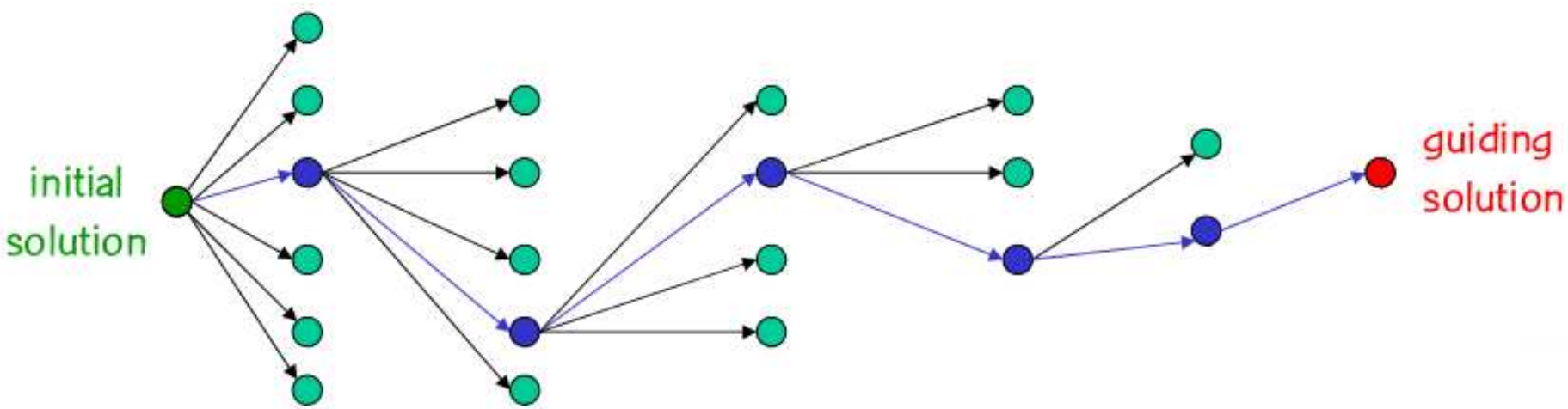


Path Relinking



Path Relinking

- Caminho é gerado selecionando movimentos que introduzem na solução inicial atributos da solução guia
- Em cada passo, todos os movimentos que incorporam atributos da solução guia são avaliados e o melhor movimento é selecionado

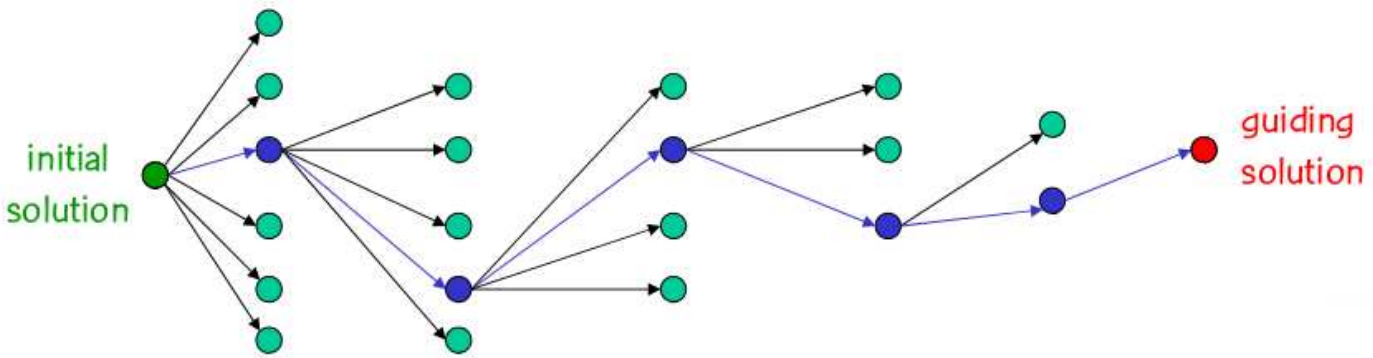


Path Relinking

- Seja $\Delta(x, y)$ a diferença simétrica entre soluções x e y a serem combinadas
- $\Delta(x, y)$ mede a quantidade de movimentos que precisam ser incluídos em x para atingir y

Algoritmo 5 PathRelinking(x, y)

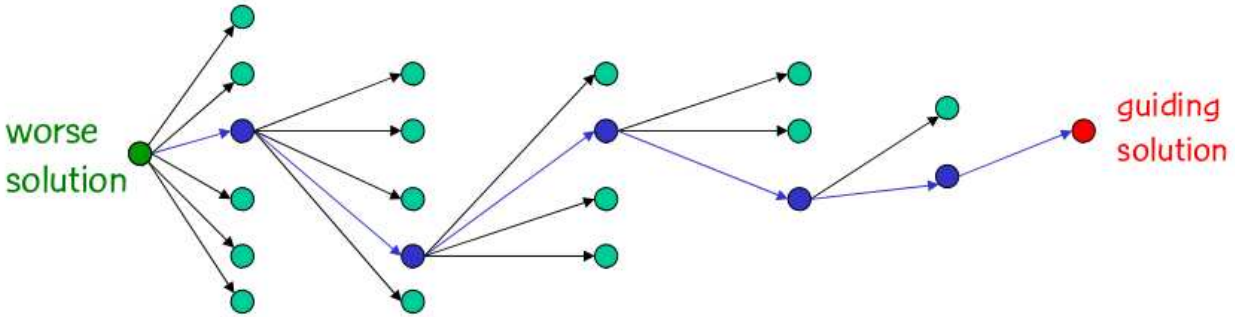
1: Calcule conjunto $\Delta(x, y)$
2: **while** ($|\Delta(x, y)| > 0$) **do**
3: Avalia movimentos correspondentes em $\Delta(x, y)$
4: Realiza melhor movimento
5: Atualiza $\Delta(x, y)$
6: **end while**



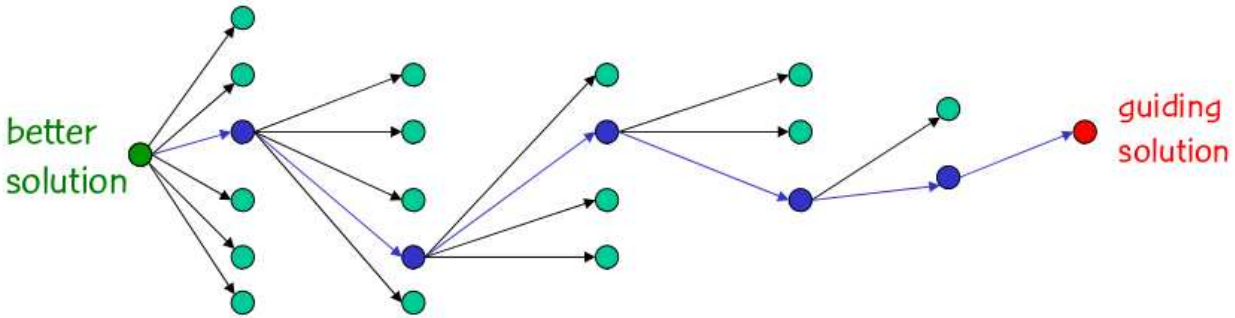
Path Relinking: Variantes

- **Variantes PR**: *trade-off* entre esforço computacional e qualidade da solução.

(1) **Forward Path Relinking**: solução inicial é a de pior qualidade entre as duas



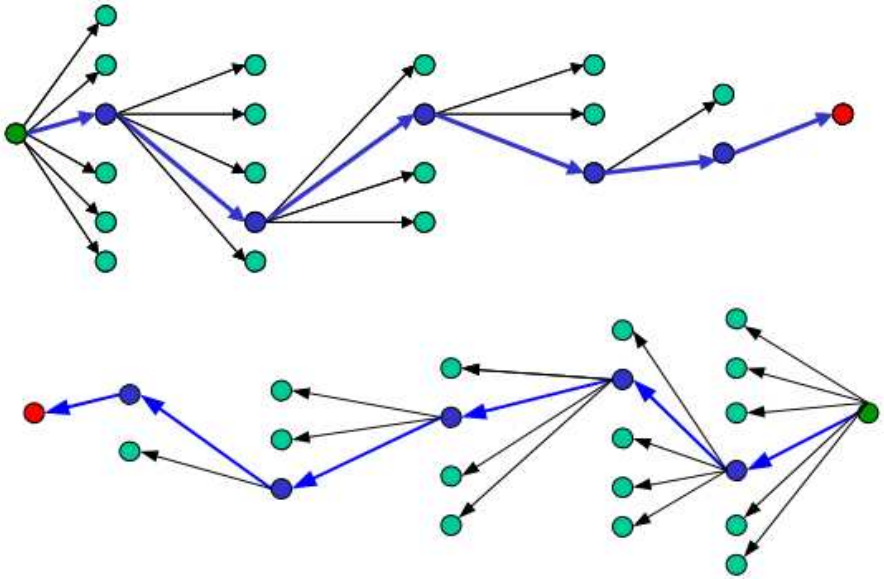
(2) **Backward Path Relinking**: solução inicial é a de melhor qualidade entre as duas



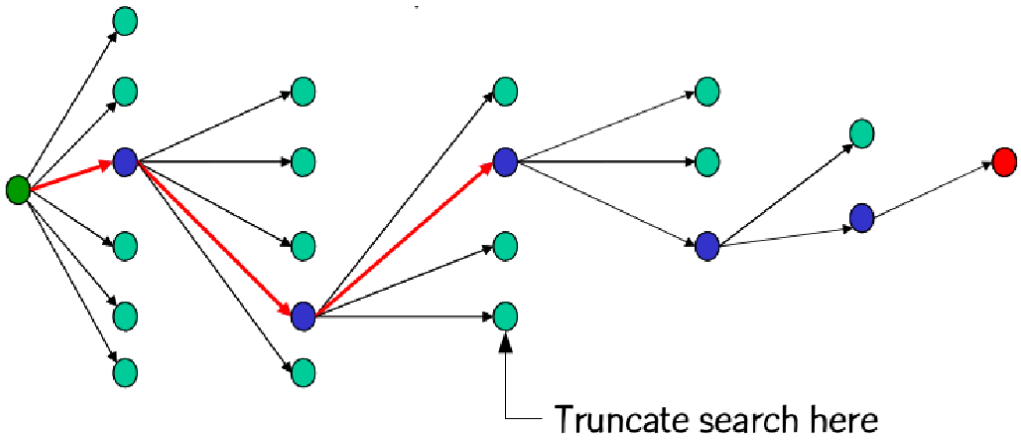
Neste caso, a vizinhança da solução inicial (i.e., melhor solução) é explorada mais do que a vizinhança da solução guia – geralmente funciona melhor

Path Relinking: Variantes

(3) **Back and Forth Path-Relinking**: explora ambas as trajetórias

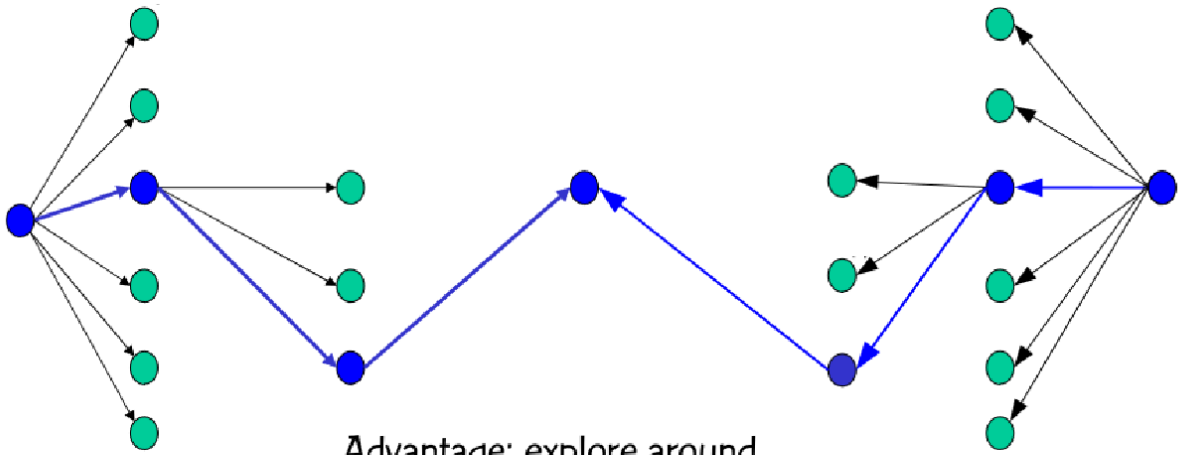


(4) **Truncated Path-Relinking**: não desenvolvendo a trajetória completa

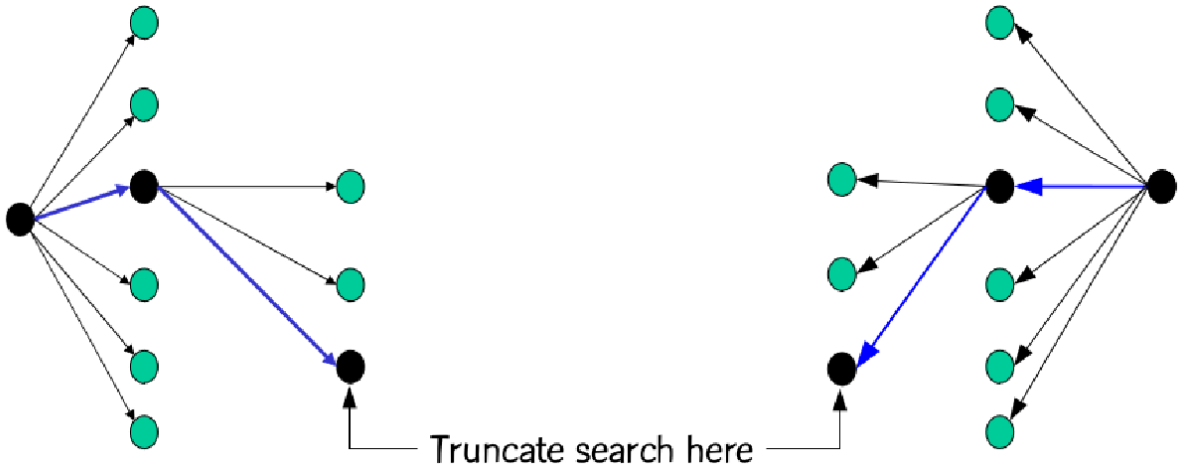


Path Relinking: Variantes

(5) **Mixed Path-Relinking**: explora vizinhança de ambas as soluções



(6) **Truncated Mixed Path-Relinking**: Truncated PR + Mixed PR



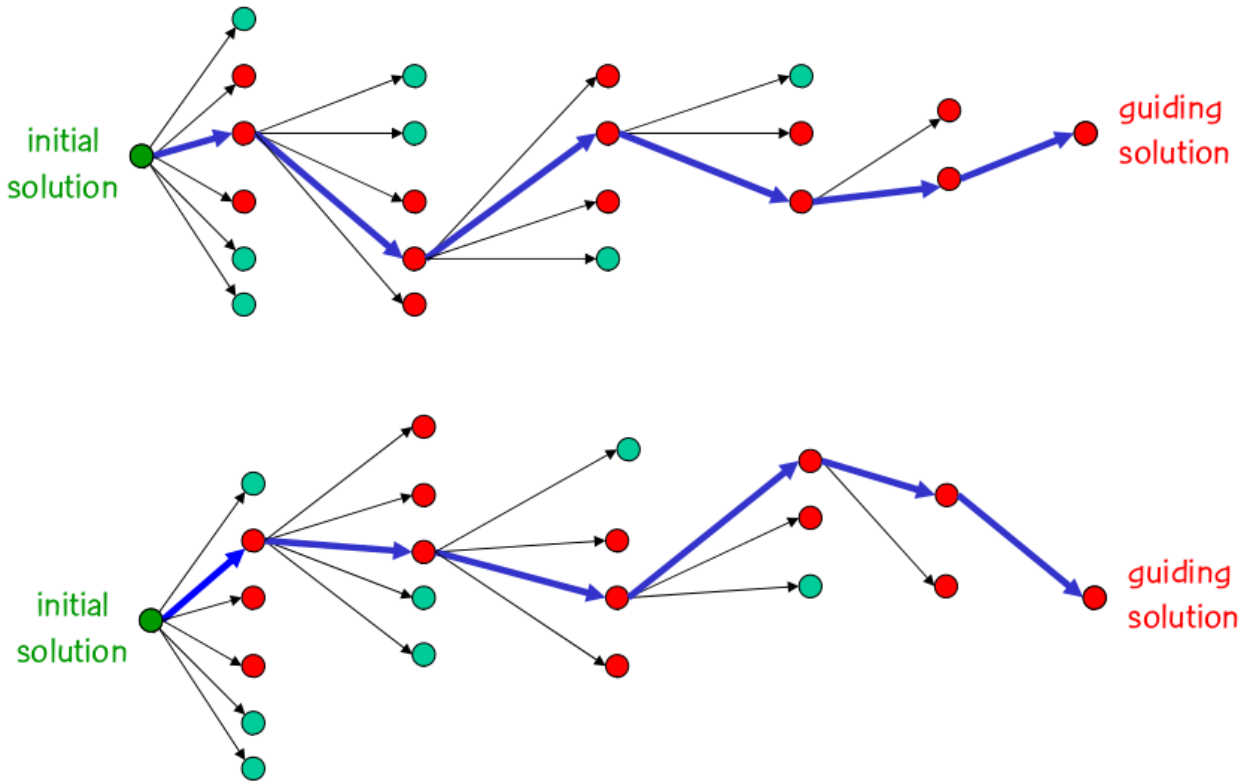
Path Relinking: Variantes

(7) **Greedy Randomized Adaptative PR**: incorpora semi-'gulosidade' no PR

Standard PR seleciona movimento de maneira gulosa: amostra apenas um dos muitos caminhos

graPR cria uma RCL com melhores movimentos: amostra diversos caminhos

Exemplo:



Path Relinking: Pseudo-Código

Algoritmo 6 PathRelinking(x_s, x_t)

1: Calcule diferenças simétricas $\Delta(x_s, x_t)$ {# *Mov. para transformar* x_s *em* x_t}
2: $f^* \leftarrow \min\{f(x_s), f(x_t)\}$
3: $x^* \leftarrow \text{argmin}\{f(x_s), f(x_t)\}$
4: $x \leftarrow x^*$
5: **while** ($\Delta(x_s, x_t) \neq \varnothing$) **do**
6: $m^* \leftarrow \text{argmin}\{f(x \oplus m) : m \in \Delta(x, x_t)\}$ {*Mov. em x com menor impacto*}
7: $\Delta(x \oplus m^*, x_t) \leftarrow \Delta(x, x_t) \setminus \{m^*\}$ {*Retira* m^* *de* Δ}
8: $x \leftarrow x \oplus m^*$ {*Aplica movimento* m^* *em x*}
9: **if** $(f(x) < f^*)$ **then**
10: $f^* \leftarrow f(x)$
11: $x^* \leftarrow x$
12: **end if**
13: **end while**
14: **return** x^*

- $\Delta(x_s, x_t)$: qtdade de movimentos necessários para alcançar x_t a partir de x_s
- $x \oplus m$: solução resultante da aplicação do movimento m à solução x

GRASP + Path Relinking

GRASP + Path-Relinking

- Path-Relinking é uma melhoria que têm sido feita no GRASP básico (desde 1999). Estudos experimentais mostraram a melhorias significativas na qualidade da solução
- Path-Relinking pode ser usado com GRASP como uma estratégia de intensificação aplicada a cada solução localmente ótima
- Duas estratégias básicas se destacam:
  (1) Etapa pós otimização aplicada à todos os pares de solução elite
  (2) Estratégia de intensificação de ótimos locais obtidos após busca local do GRASP
- Aparentemente, aplicar PR como estratégia de intensificação à cada ótimo local é mais efetivo que simplesmente utilizá-lo como passo pós-otimização
- Neste caso, PR é aplicado a pares (x_1, x_2) de soluções, onde x_1 é localmente ótima e x_2 é uma das poucas soluções elites selecionadas aleatoriamente de um *pool* de quantidade limitada **`MaxElite`**
- Cada solução localmente ótima obtida na busca local é considerada como candidata a ser inserida neste *pool* se ela for suficientemente diferente de cada outra já existente no *pool*
- Se o *pool* já está completo e a candidata é melhor do que a pior do *pool*, então a candidata substitui a pior; caso contrário ela é simplemente inserida no *pool*.

GRASP + Path-Relinking: Intensificação

- Algoritmo inicial computando diferenças simétricas $\Delta(x_1, x_2)$ entre x_1 e x_2
- Tal cômputo resulta no conjunto de movimentos que deve ser aplicado à solução inicial para atingir solução guia
- Partindo da solução inicial, o melhor movimento de $\Delta(x_1, x_2)$ ainda não realizado é aplicado à solução corrente, até atingir-se a solução guia
- A melhor solução encontrada nesta trajetória é também considerada uma candidata para inserção no *pool*, atualizando-o se necessário
- Segundo Ribeiro e Resende, em [Glover], trabalhos recentes consideram algumas alternativas para o GRASP + PR. Exemplos:
  - Não aplicar o PR a cada iteração do GRASP, mas apenas periodicamente;
  - Explorar duas trajetórias diferentes, usando primeiro x_1, seguido de x_2 como solução inicial;
  - Explorar apenas uma trajetória, iniciando ou de x_1 ou de x_2;
  - Não seguir a trajetória completa, mas apenas parte da mesma (truncar)

GRASP + Path-Relinking: Pós-Otimização

- Quando usado na pós-otimização, o PR é aplicado à uma série de *pools*
- O *pool* P_0 é o *pool* P obtido no término das iterações do GRASP
- O valor da melhor solução de P_0 é aplicado a f_o^* e o contador $k \leftarrow 0$ é inicializado
- Na k-ésima iteração, todos os pares de elementos no *pool* P_k são combinados usando PR
- Cada resultado do PR é testado para pertinência no *pool* P_{k+1} usando mesmos critérios usados no GRASP
- Se uma nova melhor solução é produzida, i.e., $f_{k+1}^* < f_k^*$, então $k \leftarrow k + 1$ e uma nova iteração do PR na pós-otimização é realizada
- Caso contrário, o PR encerra com $x^* = \text{argmin}\{f(x)|x \in P_{k+1}\}$
- Solução final do GRASP + PR é a solução x^* encontrada na pós-otimização

GRASP + Path-Relinking

- Em cada iteração da hibridização GRASP + PR ocorrerão:

  (1) **Fase Construtiva**

  Um procedimento guloso, aleatorizado e adaptativo é usado para construir uma solução factível

  (2) **Fase Buscal Local**

  A solução construída na primeira fase é melhorada progressivamente por uma estratégia de busca na vizinhança, até que um mínimo local seja encontrado

  (3) **Fase Path-Relinking**

  PR é aplicado à solução obtida na fase de busca local e à uma solução selecionada aleatoriamente do *pool*

  A melhor solução encontrada ao longo desta trajetória é também considerada como uma candidata para inserção no *pool*

- Ao término de todas as iterações do GRASP, uma fase de pós-otimização combina as soluções elite no *pool* em busca de melhores soluções.

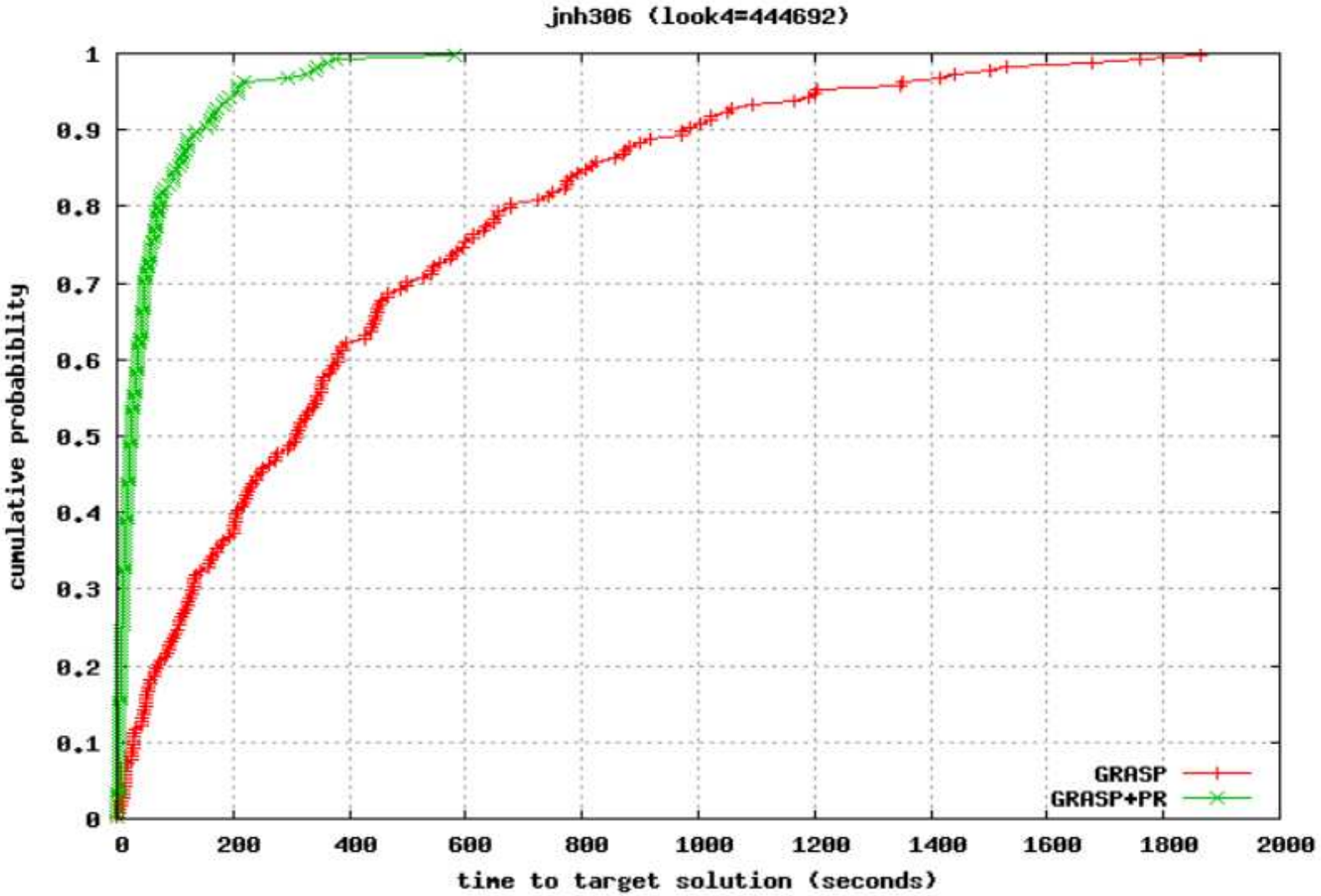
GRASP + Path-Relinking: Pseudo-Código

Algoritmo 7 GRASP-PR(*MaxIter*)

1: $P \leftarrow \varnothing$
2: $f^* \leftarrow \infty$
3: **for** $i = 1, \ldots, MaxIter$ **do**
4: $x \leftarrow$ **GreedyRandomizedConstruction()**
5: $x \leftarrow$ **LocalSearch(**x**)**
6: **if** $(i \geq 2)$ **then**
7: Escolha, aleatoriamente, soluções $\mathcal{Y} \in P$ do *pool* para religar com x
8: **for** $(y \in \mathcal{Y})$ **do**
9: Determine qual (x ou y) é a solução inicial x_s e alvo x_t
10: $x_p \leftarrow$ **PathRelinking(**x_s**,** x_t**)**
11: Atualize o conjunto elite P com candidato x_p
12: **if** $(f(x_p) < f^*)$ **then**
13: $f^* \leftarrow f(x_p)$
14: $x^* \leftarrow x_p$
15: **end if**
16: **end for**
17: **end if**
18: **end for**
19: $P \leftarrow$ **PostOptimize(**P**)**
20: $x^* \leftarrow \text{argmin}\{f(x) : x \in P\}$
21: **return** x^*

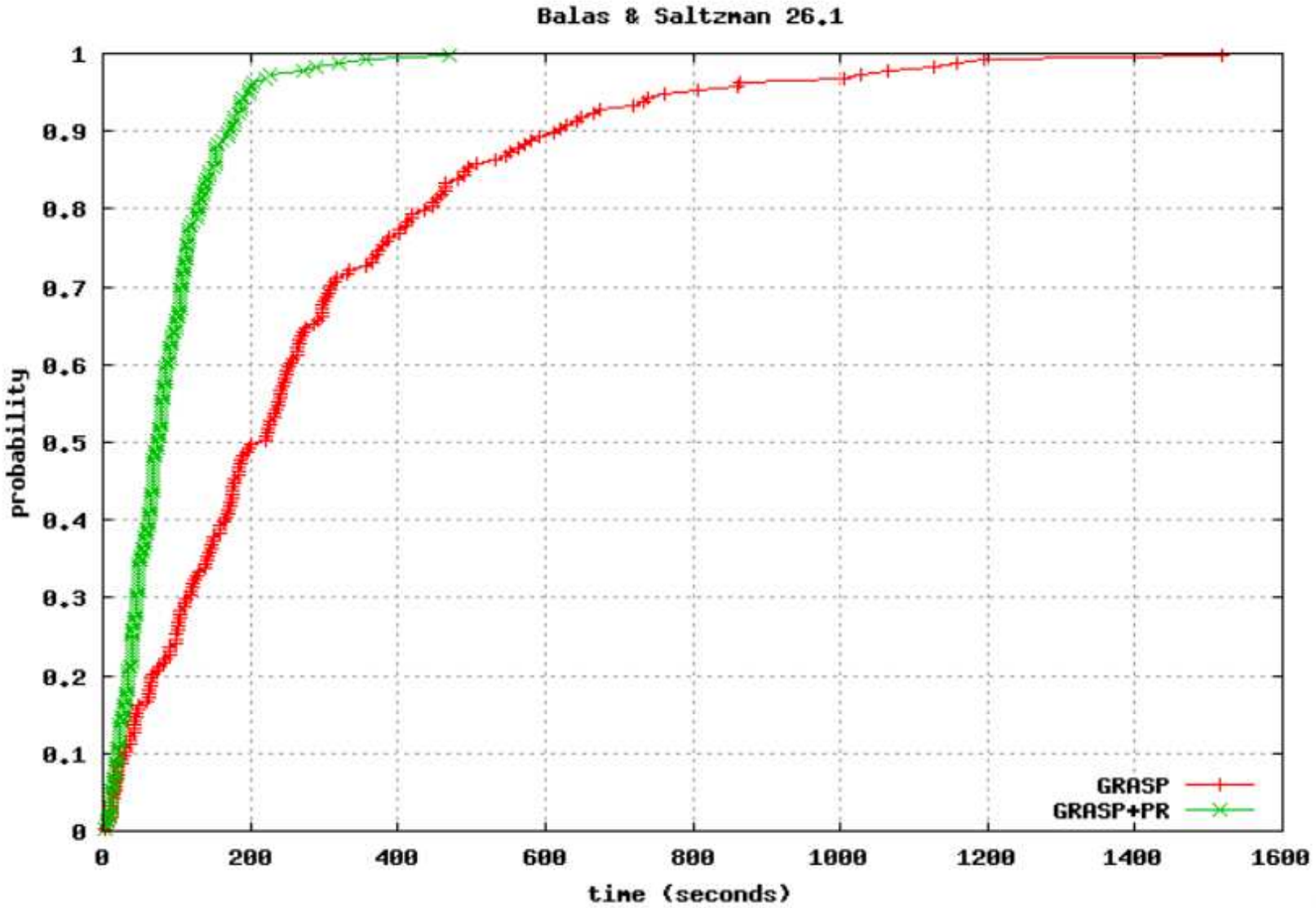
GRASP + Path-Relinking: Alguns Resultados

MAX-SAT (Festa, Pardalos, Pitsoulis, and Resende, 2006)



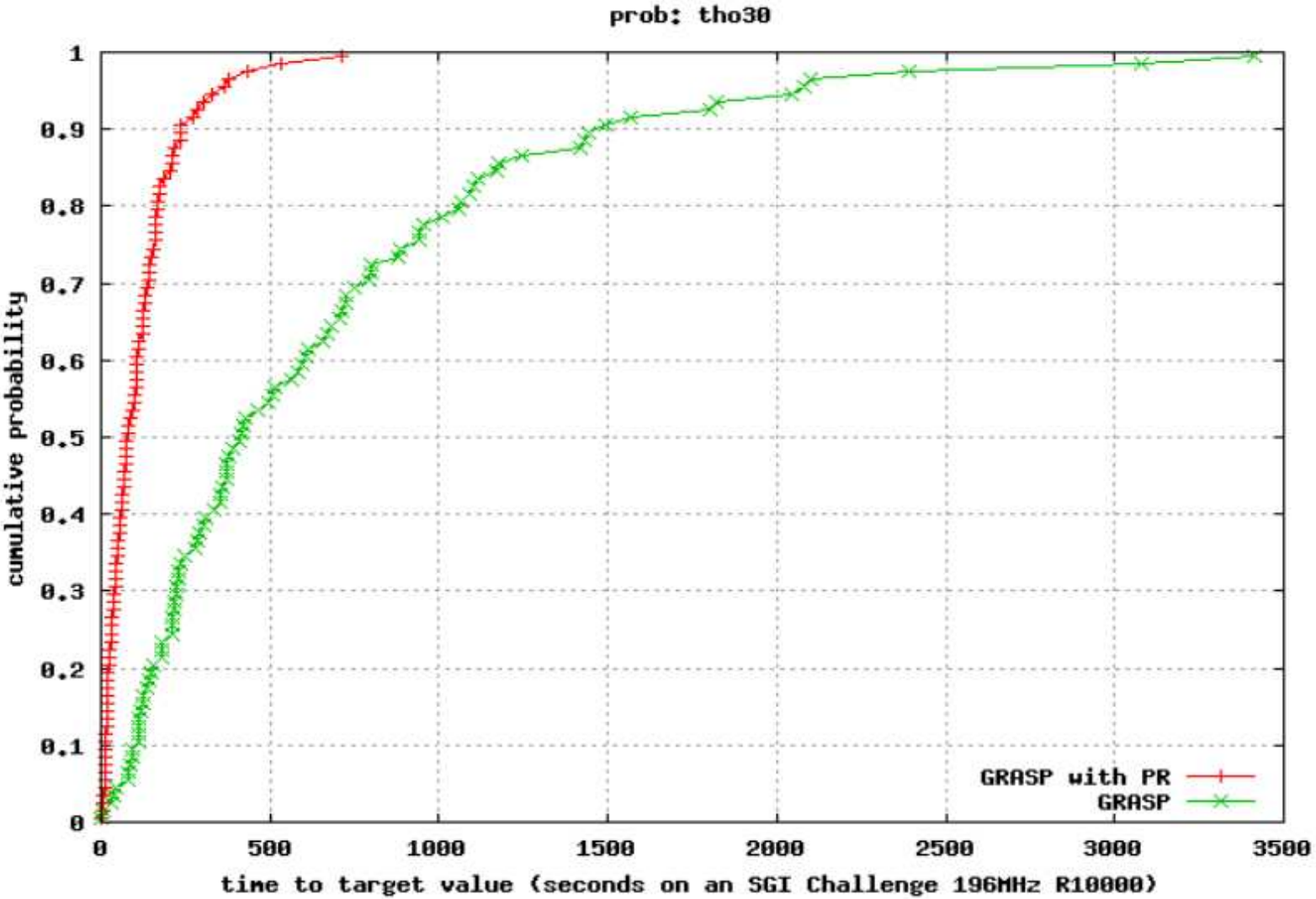
GRASP + Path-Relinking: Alguns Resultados

3-index assignment (Aiex, Resende, Pardalos, & Toraldo, 2005)



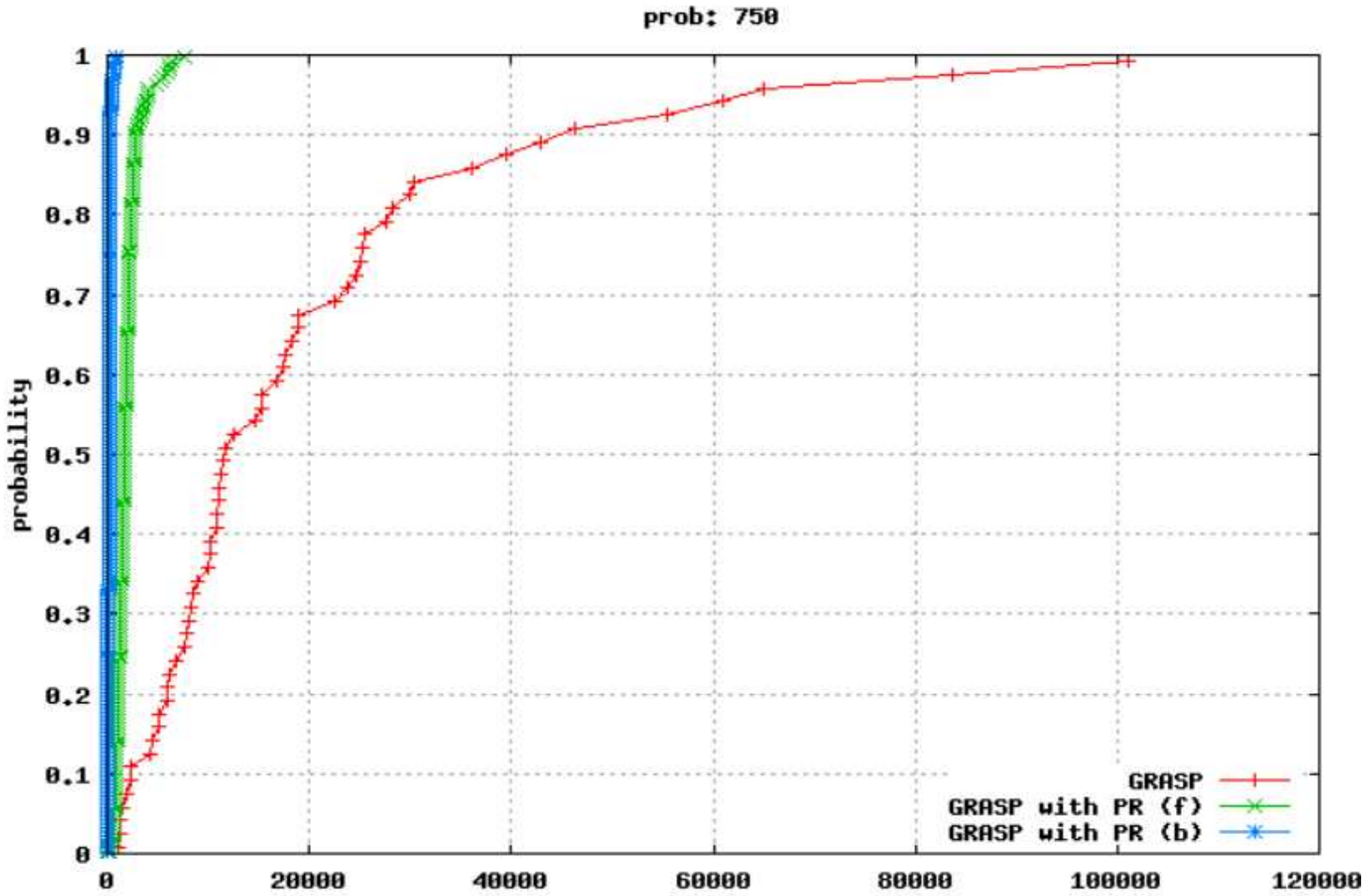
GRASP + Path-Relinking: Alguns Resultados

QAP (Oliveira, Pardalos, and Resende, 2004)



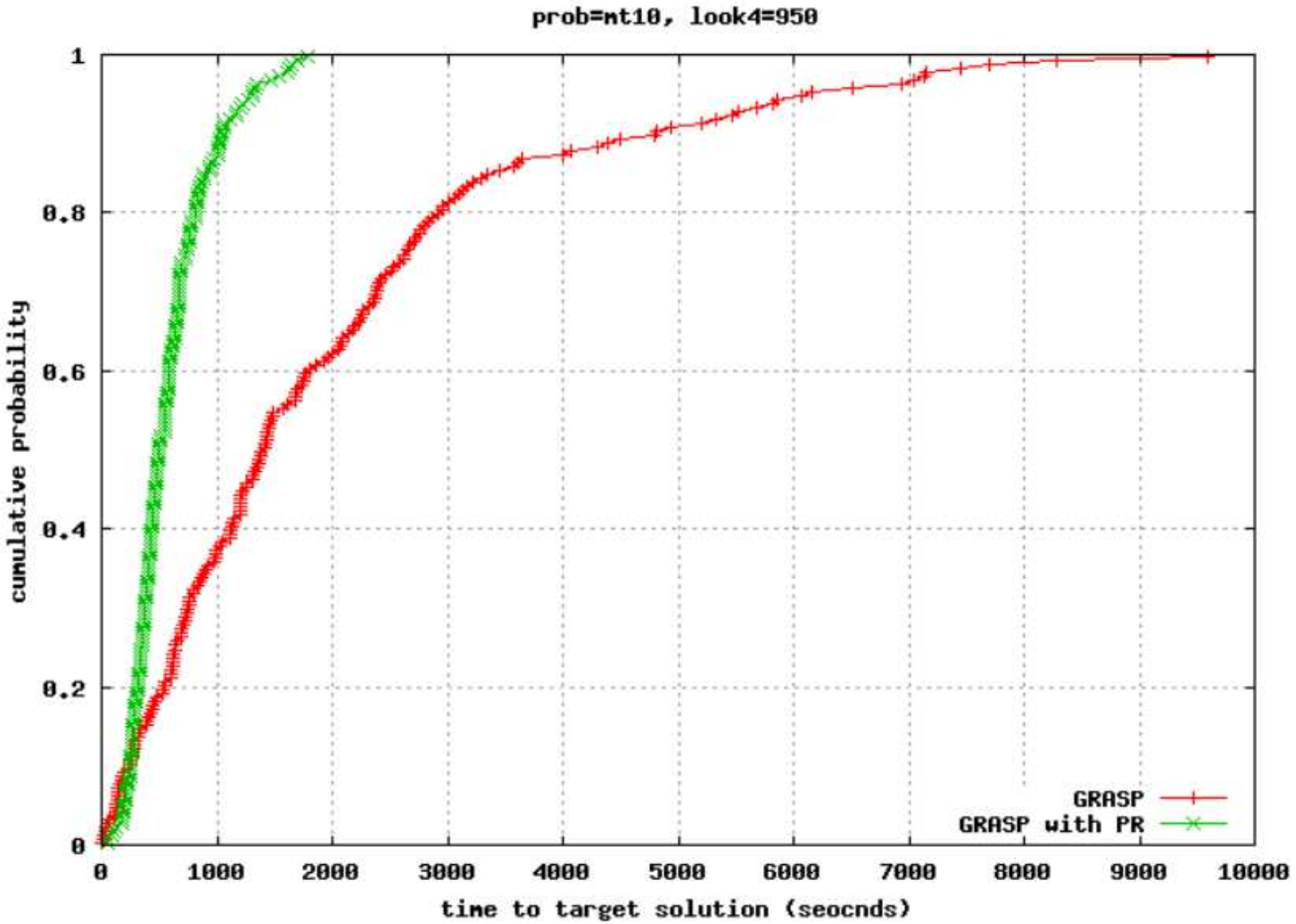
GRASP + Path-Relinking: Alguns Resultados

Bandwidth packing (Resende and Ribeiro, 2003)



GRASP + Path-Relinking: Alguns Resultados

Job shop scheduling (Aiex, Binato, & Resende, 2003)



Referências Bibliográficas

Referências Bibliográficas


CAMPELLO R. E, MACULAN N.
Algoritmos e Heuristicas: Desenvolvimento, Avaliação e Performance.
Editora da Universidade Federal Fluminense.


NETTO P. O. B.
Grafos: Teoria, Modelos e Algoritmos, 4a. edição.
Editora Blucher. ISBN: 9788521203919


GLOVER F., KOCHENBERG G. A.
Handbook of Metaheuristics.
Editora Springer. ISBN: 1-4020-7263-5


GAREY M. R., JOHNSON D. S.
Computers and Intractability - A Guide to the Theory of NP-Completeness.
Editora Freeman and Company.


GOLDBARG M. C., LUNA H. P.
Otimização Combinatória e Programação Linear: Modelos e Algoritmos.
Editora Campus.


EIBEN A., SMITH J.
Introduction to Evolutionary Computing.
Editora Springer (Natural Computing Series). ISBN: 3540401849.


PARDALOS P., RESENDE M. G.
Handbook of Applied Optimization.
Editora Oxford.

Referências Bibliográficas



RIVEST R. L., LEIRSON C. E., CORMEN, T. H., STEIN, C.
Algoritmos: Teoria e Prática, 3a. edição.
Editora Campus. ISBN: 9788535236996



LOPES, H. S., RODRIGUES L. C. A., STEINER M. T. A.
Meta-heurísticas em Pesquisa Operacional
Editora Ominipax. ISBN: 978-85-64619-10-4 [recurso eletrônico]
DOI: 10.7436/2013.mhpo.0



RUSSEL, S., NORVIG, P.
Inteligência Artificial, 2a. Edição.
Editora Elsevier, 2004. ISBN: 978-85-352-1177-1